

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 25

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Arunprasath R (192524077) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Arunprasath R

Asunprasath . R
192524077

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:

id + id

Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:

$S \rightarrow aS \mid b$

Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:

$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:

$S \rightarrow aA \mid b$

$A \rightarrow c \mid \epsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.

$S \rightarrow Sa \mid b$

Justify your answer.

Q6. Apply left factoring to the grammar:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Q7. Construct a parse tree for the string:

a + b

using the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q8. Write the recursive descent procedures for the grammar:-

$S \rightarrow aA$

$A \rightarrow b$

Q9. Given the grammar:-

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:

id + id

Q11. Identify the handle in the string:

id + id

for the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q12. Construct the operator precedence relations for the operators:

+

*

where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:

id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:

$S \rightarrow aA$
 $A \rightarrow b$

Q15. Given the grammar➤

$S \rightarrow aA$
 $A \rightarrow b$

Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

Q17. Construct the augmented grammar for:

$S \rightarrow a$

Q18. Find the closure of the LR(0) item:

$S' \rightarrow \cdot S$
for the grammar:
 $S \rightarrow a$

Q19. Check whether the grammar is ambiguous:

$E \rightarrow E + E$
 $E \rightarrow id$
using the string:
 $id + id + id$

Q20. Construct a leftmost derivation for the string:

$id + id$
using the grammar:
 $E \rightarrow E + E$
 $E \rightarrow id$

Q21. Apply left factoring to the grammar:

$S \rightarrow aA$
 $S \rightarrow aB$
 $A \rightarrow b$
 $B \rightarrow c$

Q22. Find FIRST(A) for the grammar:

$A \rightarrow x$
 $A \rightarrow y$

Q23. Determine whether the grammar is suitable for predictive parsing.

$S \rightarrow aA \mid aB$
 $A \rightarrow b$
 $B \rightarrow c$

Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:

$S \rightarrow aS$
 $S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:

$S \rightarrow aS$
 $S \rightarrow b$
Show the table entries for the terminals:
 $a, b, \$$

C02 AT2.

Asunprasath. R

192524077

Compiler Design

Short Answer test

08/08/2026.

Asunprasath R

192524077

Compiler Design

8/8/26.

Short Answer test

1)

Token stream: $id + id$

- Parser checks tokens against grammar
(Ex: $E \rightarrow E + E \mid id$)
- id matches E , $+$ is valid operator & second id also matches E .
- Expression is syntactically valid.

2)

$S \rightarrow aS/b$

$S \rightarrow aS$

$\rightarrow aas$

$\rightarrow aaas$

$\rightarrow aaab$

3)

$A \rightarrow Aa/b$

$A \rightarrow bA'$

$A' \rightarrow aA' \mid \epsilon$

4)

$S \rightarrow aA/b$

$A \rightarrow c/\epsilon$

$FIRST(S) = \{a, b\}$

5)

$S \rightarrow Sa/b$

No, it contains left recursion, which causes infinite recursion in top-down parsers.

6)

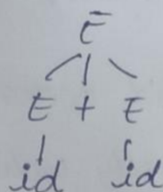
$E \rightarrow \text{if } E \text{ then } S$
 $S \rightarrow \text{if } E \text{ then } S \text{ else } S.$

After left factoring:

$S \rightarrow \text{if } E \text{ then } SS'$
 $S' \rightarrow \text{else } S/\epsilon.$

7)

$E \rightarrow E + E$
 $E \rightarrow \text{id}$



8)

$S \rightarrow aA$
 $A \rightarrow b.$

void S()

{

match('a');

A();

}

void A()

{ match('b');

}

9)

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

$FIRST(B) = \{b\}$

$FIRST(A) = \{a\}$

$FIRST(S) = \{a\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{b\}$

$FOLLOW(B) = \{\$ \}$

Non-Terminal

S

A

B

a

$S \rightarrow AB$

$A \rightarrow a$

~~$B \rightarrow b$~~ $B \rightarrow b$

b

10)

Input: id + id

1. Shift id
2. Reduce id $\rightarrow E$
3. Shift +.

$E \rightarrow E + E$

$E \rightarrow id$

string: $id + id$

first handle is: id .

12) Higher precedence: $*$

Relations:

$+ < *$				$+$	$($	$*$
$* > +$				$*$	$)$	$+$
$+ = +$	$+$	$<$	$>$	$+$	$=$	$+$
$* = *$	$*$	$>$	$>$	$*$	$=$	$*$

13) Exp: $id * id$.

steps:-

shift first id

shift $*$

shift second id

since $*$ has highest precedence, reduce $id * id$ first.

14) An LR parser scans input from left to right & produces the rightmost derivation in reverse.

$S \rightarrow aA$

$A \rightarrow b$

Parsing: Action

shift a

shift b

reduce $A \rightarrow b$

reduce $S \rightarrow aA$

Accept

stack

$\$a$

$\$ab$

$\$aA$

$\$S$

$\$S$

Input

$b\$$

$\$$

$\$$

$\$$

$\$$

16) Grammar: $S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

since A is followed by B ,

$FOLLOW(A) = FIRST(B) = \{b\}$

- 15)
1. Augment the grammar
 2. Construct LR(0) item sets
 3. compare closure & goto.
 4. Compute FIRST & FOLLOW.
 5. Fill ACTION table.
 6. Fill GOTO table.

$$\begin{aligned} \phi &\rightarrow aA \\ A &\rightarrow b. \end{aligned}$$

$$\begin{aligned} S &\rightarrow S' \\ S &\rightarrow aA \\ A &\rightarrow b \end{aligned}$$

$$\begin{aligned} S &\rightarrow \cdot S' \\ S &\rightarrow \cdot aA \\ A &\rightarrow \cdot b. \end{aligned}$$

17) $S \rightarrow a$

Augmented :-

$$\begin{aligned} S' &\rightarrow S \\ \phi &\rightarrow a. \end{aligned}$$

18) closure of $S' \rightarrow * \phi$

G: $S \rightarrow a$

closure: $\begin{aligned} S' &\rightarrow * S \\ S &\rightarrow * a. \end{aligned}$

19) Ambiguity:

G: $\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow id \end{aligned}$

String: $id + id + id$

The grammar is ambiguous because it has 2 different parse trees:

- i) $(id + id) + id$
- ii) $id + (id + id)$

20) G: $\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow id \end{aligned}$

$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow id + E \\ &\Rightarrow id + id. \end{aligned}$$

original:

$S \rightarrow aA$

$S \rightarrow aB$

$A \rightarrow b$

$B \rightarrow c$

Factored Grammar:-

$S \rightarrow aX$

$X \rightarrow A|B$

$A \rightarrow b$

$B \rightarrow c$

22) $FIRST(A)$

Grammar:- $A \rightarrow x$
 $A \rightarrow y$

$FIRST(A) = \{x, y\}$

23)

G:- $S \rightarrow aA/aB$
 $A \rightarrow b$
 $B \rightarrow c$

NO, REASON: $FIRST(aA) = FIRST(aB) = \{a\}$

$FIRST$ sets overlap, causing a parsing table conflicts.

24)

$S \rightarrow aS$

$S \rightarrow b$

void S()

{ if (lookahead == 'a')

{ match('a');

S();

}

else if (lookahead == 'b')

{ match('b');

}

25) Q:

$S \rightarrow aS$

$S \rightarrow b$

$N-T$	a	b	$\$$
S	$S \rightarrow aS$	$S \rightarrow b$	Error.

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand